DISTRIBUTED ALGORITHMS • CONSENSUS & TRANSACTIONS • ARCHITECTURAL DISSECTION

Distributed Transactions: Two-Phase Commit Protocol (2PC)

Synthesis based on Arpit Bhayani's Distributed Systems Masterclass • Zomato Instant 10-Minute Architecture

EXECUTIVE SUMMARY & FUNDAMENTAL DISTRIBUTED INVARIANT

Atomic transactions across autonomous, network-separated services require an all-or-nothing guarantee without shared physical memory. Two-Phase Commit (2PC) splits commitment into Phase 1 (Prepare / Voting) and Phase 2 (Commit / Abort), enforcing the absolute invariant: *A transaction commits if and only if 100% of participating cohorts vote YES*. If even a single node votes NO or experiences network timeout, the Coordinator aborts globally.

1. REAL-WORLD CASE STUDY: ZOMATO INSTANT 10-MINUTE DELIVERY

In ultrafast food delivery (Zomato Instant), atomic multi-party synchronization is a prerequisite for customer fulfillment. When an order is placed, three distinct services must commit simultaneously within milliseconds:

1. Order Core Service

Validates customer credit / payment authorization, generates the canonical Order ID, and establishes billing audit lines.

2. Dark Kitchen Service

Checks physical finished-goods inventory in the localized micro-hub, locking 1 portion of perishable food stock.

3. Delivery Fleet Service

Scans GPS telemetry to locate and lock an active delivery partner within a strict 1.5 km radial bounding box.

2. FAILURE MODES OF NAIVE MICROSERVICE CHAINING

Linear sequential chaining (Service A → Service B → Service C) collapses under real-world network and hardware failures:

- **Compensating Transaction Failure:** If the Dark Kitchen reserves biryani, but no delivery rider is found, the caller issues a compensating `release_food()` RPC. If a network blip or GC pause drops that packet, the kitchen remains locked permanently.
- **Mid-Flight Coordinator Crash:** If the coordinating process terminates after charging the user but before rider dispatch, money is captured with zero fulfillment.
- **Dirty Read Anomalies:** Intermediate mutations are visible to concurrent shoppers, leading to phantom inventory depletion.

3. TWO-PHASE COMMIT PROTOCOL SPECIFICATION

2PC introduces a dedicated Coordinator and assigns participating nodes as Cohorts (Resource Managers):

Phase 1: Prepare (Voting Phase)

Coordinator writes `START_2PC` to disk and sends `PREPARE` to all cohorts. Each cohort checks constraints, acquires exclusive row locks, writes undo/redo logs to WAL, and replies `VOTE_COMMIT` (YES) or `VOTE_ABORT` (NO).

Phase 2: Commit (Decision Phase)

If ALL cohorts voted YES, Coordinator flushes `GLOBAL_COMMIT` to disk and broadcasts commit. If ANY cohort voted NO or timed out, Coordinator flushes `GLOBAL_ABORT` to disk and broadcasts abort.

4. PROTOCOL MECHANICS: THE TWO PHASES IN DETAIL

The sequence of operations enforces durability and lock isolation across every participant:

Phase 1: The Binding Readiness Contract

1. **Coordinator Dispatch:** Coordinator logs transaction state and sets timer $\tau_{prepare}$. Broadcasts `PREPARE(TxID)`.
2. **Cohort Lock Acquisition:** Cohort acquires exclusive locks on affected database rows (e.g., `SELECT ... FOR UPDATE`).
3. **Cohort WAL Force:** Cohort writes undo log (to revert changes) and redo log (to apply changes) to durable disk storage.
4. **The Irrevocable Promise:** If successful, Cohort writes `PREPARED` to disk WAL and responds with `VOTE_COMMIT`. Once this vote is sent, the cohort *surrenders the right to abort unilaterally*.

Phase 2: Final Verdict & Lock Release

1. **Decision Point:** If all votes are YES, Coordinator flushes `GLOBAL_COMMIT` to disk. The exact instant that disk block hits non-volatile storage, the transaction is legally committed.
2. **Decision Broadcast:** Coordinator sends `GLOBAL_COMMIT` (or `GLOBAL_ABORT`) to all cohorts.
3. **Local Application:** Cohorts apply mutations to base tables, release all acquired locks, write `END` to local WAL, and send `ACK`.
4. **Garbage Collection:** Coordinator receives all ACKs, writes `END_2PC`, and frees in-memory state.

5. FAILURE MODES & CRASH RECOVERY MATRIX

Failure Point	Detection Mechanism	Recovery Action & Protocol Guarantee
Cohort crashes before voting in Phase 1	Coordinator timer $\tau_{prepare}$ expires	Coordinator aborts transaction globally. When cohort reboots, it finds no commit decision and aborts locally. Safe.
Cohort crashes after voting VOTE_COMMIT	Cohort on reboot scan	Cohort finds `PREPARED` in WAL. It must query the Coordinator for the final verdict before releasing locks.
Coordinator crashes after receiving all YES votes	Cohorts timer τ_{cohort} expires	Cohorts are stuck in `PREPARED` state. They hold locks indefinitely. This is the **Coordinator Blocking Problem**.
Coordinator crashes after logging GLOBAL_COMMIT	Coordinator on reboot scan	Coordinator scans WAL, sees `GLOBAL_COMMIT` without `END_2PC`. Re-broadcasts `GLOBAL_COMMIT` to all cohorts.

6. THE COORDINATOR BLOCKING DILEMMA (THE FUNDAMENTAL FLAW)

Because 2PC is a **blocking consensus protocol**, the failure of the Coordinator while cohorts are prepared causes complete paralysis:

- Cohorts cannot unilaterally commit, because the Coordinator might have issued an abort due to another cohort timing out.
- Cohorts cannot unilaterally abort, because the Coordinator might have committed and notified another node before dying.
- Consequently, database rows remain locked, blocking subsequent operations and starving the entire system until the coordinator recovers.

7. COMPLEXITY, LATENCY & PERFORMANCE BOUNDS

The mathematical and operational overhead of Two-Phase Commit imposes strict throughput limits:

Message Complexity

For K cohorts: Phase 1 requires $2K$ messages (Prepare + Vote). Phase 2 requires $2K$ messages (Decision + ACK). Total network messages = $\mathcal{O}(4K)$ per transaction.

Commit Latency Bound

Latency requires at least 2 network Round Trip Times (RTT) plus 2 synchronous disk `fsync` writes: $ ext{Latency} \ge 2 imes ext{RTT} + 2 imes T_{ ext{fsync}} pprox 8 ext{ to }15 ext{ ms}$.

The Convoy Effect & Straggler Penalty

Because exclusive locks are held continuously from Phase 1 vote until Phase 2 ACK, the duration of data lockup is dictated by the *slowest cohort in the cluster*. A single lagging cohort with high disk queue depth or network jitter degrades cluster-wide throughput.

8. PRODUCTION COMPARISON: 2PC VS. 3PC VS. SAGA

Architecture Pattern	Consistency Level	Lock Duration	Availability Under Failure	Best Use Case
Two-Phase Commit (2PC)	Strict ACID (Serial)	Duration of 2 RTTs + fsync	Blocks if Coordinator crashes	Sub-second sharded database transactions
Three-Phase Commit (3PC)	Strict ACID (Non-blocking)	Duration of 3 RTTs	Vulnerable to network partitions	Synchronous networks with no partitions
Saga Pattern (Choreography/Orch)	Eventual Consistency	Zero global locks (local only)	High availability; relies on compensation	Long-running microservices & external APIs
Paxos/Raft + 2PC (Spanner)	Strict External Consistency	Hardware-optimized TrueTime	High; consensus group replaces single node	Global-scale distributed SQL databases

9. ARCHITECTURAL ENGINEERING TAKEAWAYS

1. **Fsync Before Network Reply:** A cohort must never transmit `VOTE_COMMIT` before its undo/redo logs and transaction state are physically written to non-volatile disk. Failure to fsync can lead to phantom commits upon reboot.
2. **Strict Timeouts Prevent Zombie Locks:** Coordinators must enforce aggressive prepare timeouts. If a cohort does not respond within the deadline, the coordinator immediately aborts to free other cohorts.
3. **Avoid 2PC Across Microservice Boundaries:** Synchronous distributed locks across autonomous team services create tight coupling and cascading failure domains. Reserve 2PC for storage engine partitions and use Sagas for business services.
4. **Replicate the Coordinator:** In modern architectures (Google Spanner, CockroachDB), the Coordinator is not a single process; it is a replicated Paxos/Raft group that survives arbitrary node crashes without blocking.